Resource Management and Fairness for Federated Learning over Wireless Edge Networks

Ravikumar Balakrishnan, Mustafa Akdeniz, Sagar Dhakal, Nageen Himayat
Intel Labs, USA, {*ravikumar.balakrishnan, mustafa.akdeniz, sagar.dhakal, nageen.himayat*}*@intel.com*

***Abstract*—Federated Learning has the potential to break the barrier of AI adoption at the edge through better data privacy and reduced client to server communication cost. However, the heterogeneity among the clients' compute capabilities, communication rates, the amount and quality of data can affect the training performance in terms of overall accuracy, model fairness and convergence time. We develop compute-communication and data importance aware resource management schemes to optimize the above metrics and evaluate the training performance on benchmark datasets. We observe that the proposed algorithms strikes a balance between model performance and total training time by achieving 4x - 10x reduction in convergence time without loss of test performance. Further, our algorithms also show superior fairness performance measured by variance and worst case 10^{th} percentile accuracy/loss on benchmark datasets.**

***Index Terms*—Federated learning, convergence, fairness**

I. Introduction

Building machine learning (ML) models from massive amounts of user-generated data is paramount in this age of information. It is well-known that data is generated across a wide range of devices including mobile phones, sensors, wearable devices, autonomous vehicles. Training ML models directly at client devices instead of a cloud-based ML training has significant advantages such as improved data privacy, reduction in data storage, maintenance at cloud servers as well as efficient use of growing compute capabilities of client devices. Hence, distributed learning over a range of connected devices at the edge network has gained significant momentum.

Federated learning (FL) [1] is a distributed ML approach where a federation of clients jointly learn a model by only sharing model specific parameters with a Mobile Edge Computing/MEC server over a communication network. FL can be applied to scenarios with a set of clients with local training dataset and computational resource to perform training. For example, models for face recognition, language models for text autocompletion and voice recognition can be learnt by a fleet of smartphones. FL can also be utilized for sensor/IoT devices including wearables, surveillance cameras or autonomous vehicles where data privacy is important.

While FL has the potential to accelerate AI adoption at the edge, it faces several challenges: a) *Compute Heterogeneity:* The compute capability of clients including available cycles may be highly varying, resulting in delayed or even missing model updates which can the global model; b) *Communication Heterogeneity:* Clients may support different communication rates depending on their wireless links to the MEC server causing straggler/slow devices. The communication rates may also be time-varying leading to more complex straggler dynamics; c) *Statistical Heterogeneity*, when data from different clients may not follow the true distribution of the overall data, thus violating the independent and identically distributed (I.I.D) assumption across distributed data. Moreover, the different clients may also have different number of training examples which may lead to variance in the updates across the clients. Therefore, learning a timely ML model with high accuracy is critical for time-sensitive applications such as autonomous vehicles or factory automation. It is also important to realize model fairness where the global model trained must have comparable performance across the clients. This can be viewed as a resource management problem where an MEC server orchestrates the training to optimize the performance metrics.

Resource management for FL is becoming an active research area. Prior work includes deadline-aware client selection under I.I.D data setting [2], wireless radio resource allocation and user selection [3], [4]. A FedProx algorithm was proposed in [5] to address statistical heterogeneity between clients but the compute and communication heterogeneity are not considered. The paper [6] deals with non-I.I.D data case as well as allocating wireless resources to clients to reduce convergence time. However, allocating radio resources to control the learning is not the focus of our work. Finally, a q-fair federated learning approach is proposed in [7] with a modified local loss functions to facilitate fair learning. Coded Federated Learning was developed in [8] to address the unreliability and compute limitations in wireless edge networks.

This paper proposes data-importance and compute-communication aware resource management algorithms in order to optimize training accuracy, fairness and convergence time. The paper is organized into the following sections. Section II formalizes the resource management problem of federated learning under compute and communication resource constraints. Section III describes the proposed resource management algorithms. Section IV outlines the experimental results on benchmark datasets with wireless and compute models. The conclusions are presented in Section V.

II. Resource Management in Federated Learning

Suppose $f(\mathbf{w})$ is cost function defined as the expected loss with model parameter $\mathbf{w}$ over training data drawn from an empirical distribution $\hat{p}_{data}$.

$$f(\mathbf{w}) = E_{\mathbf{x},\mathbf{y}\sim\hat{p}_{data}} L(\mathbf{x},\mathbf{y};\mathbf{w}) = \frac{1}{m}\sum_{i=1}^{m} L(\mathbf{x}(i), y(i); \mathbf{w}), \quad (1)$$

978-1-7281-5478-7/20/$31.00 ©2020 IEEE

$L(\mathbf{x}(i), y(i); \mathbf{w})$ can be the cross-entropy loss of model $\mathbf{w}$ on training example i. Since data is distributed in FL, the new cost function is the weighted sum of losses at the clients

$$f(\mathbf{w}) = \frac{1}{\sum_{k=1}^{N} n_k} \sum_{k=1}^{N} n_k F_k(\mathbf{w}), \tag{2}$$

$$F_k(\mathbf{w}) = \frac{1}{n_k} \sum_{i=1}^{n_k} L(\mathbf{x}(i), y(i); \mathbf{w}). \tag{3}$$

where F_k is the loss function computed on client k's n_k training examples. Each client uses a gradient-based algorithm like stochastic gradient descent (SGD) to compute local gradient $\mathbf{g}_k$ and update local weight as $\mathbf{w}_{k,t+1}$ with learning rate η.

$$\mathbf{g}_{k,t+1} = \nabla_{\mathbf{w}} F_k(\mathbf{w}) = \frac{1}{n_k} \sum_{i=1}^{n_k} \nabla_{\mathbf{w}} L(\mathbf{x}(i), y(i); \mathbf{w}), \tag{4}$$

$$\mathbf{w}_{k,t+1} = \mathbf{w}_t - \eta \mathbf{g}_{k,t+1}. \tag{5}$$

The MEC server combines the weights from the clients to obtain the global weights for next round as

$$\mathbf{w}_{t+1} = \frac{1}{\sum_{k=1}^{N} n_k} \sum_{k=1}^{N} n_k \mathbf{w}_{k,t+1}. \tag{6}$$

Since federated learning utilizes synchronous weight updates as in Eq. 6, it is inefficient to aggregate weight updates from all clients in each global update round. When a subset of K clients participate in each training round (similar to performing SGD on a mini-batch), the weight update in each round is given as

$$\mathbf{w}_{t+1} = \frac{1}{\sum_{k=1}^{K} n_k} \sum_{k=1}^{K} n_k \mathbf{w}_{k,t+1}. \tag{7}$$

FL also extends this approach to allow clients to run τ local gradient updates at each client before performing a global update step, we refer to this baseline as FedAvg algorithm.

While this approach relies on the I.I.D data assumption, the client data is likely non-identical leading to variance in gradient/weight estimates from clients. Moreover, the number of training examples across clients varies, also leading to variance in gradient/weight estimates. Finally, when clients have heterogeneous compute and communication rates, sampling clients randomly will lead to straggler issues and significantly increases the convergence time.

We focus on the resource management problem of sampling clients in each round taking into account communication, compute and statistical heterogeneity with the goal of reducing convergence time as shown in Figure 1. Selecting K out of N clients is a solution to the stochastic optimization objective

$$\min_{\mathbf{a}*} \frac{1}{\sum_{k=1}^{N} n_k} \sum_{k=1}^{N} n_k F_k\left(\frac{1}{\sum_{k=1}^{N} a_k n_k} \sum_{k=1}^{N} a_k n_k \mathbf{w}_{k,t+\tau}\right), \tag{8}$$

$$subject\ to\ P[\max(a_k T_k^{up}) > T^{max}] <= \gamma; \sum_{k=1}^{N} a_k = K, \tag{9}$$

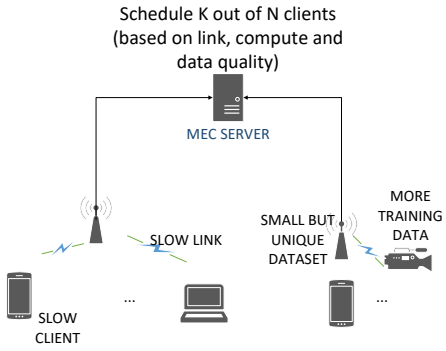


Fig. 1: Resource Management in Federated Learning

The target function is the average weighted loss over all clients where client selection vector **a** is the binary optimization vector with length N and exactly K non-zero entries. The selected clients are assumed to have computed τ local gradient steps to return the weights $\mathbf{w}_{k,t+\tau}$ to the server. T_k^{up} is the upload time for weights from client k. T^{max} is the maximum allowed time for each global round and γ indicates the maximum allowed delay violation probability for each global round.

III. Proposed Resource Management Framework for Federated Learning

The optimization framework in Eq.8 and 9 is a combinatorial problem which is NP-hard. Therefore, we utilize importance sampling approach to approximately solve this problem. To this end, the loss $L(\mathbf{x}(i), y(i); \mathbf{w})$ corresponding to each training example provides a measure of the dissimilarity of the classifier from the true label and hence is utilized in approaches to accelerate learning. Specifically, sampling a training example based on the loss/error distribution of all examples has shown to provide training speed up in [10].

A. Approach 1: Importance Sampling for Federated Learning

We utilize this intuition for the case of federated learning where, instead of looking at the loss over each example, we define a probability distribution q over the weighted loss over the clients and sample clients from this distribution.

$$\bar{q}_k = n_k F_k(\mathbf{w}_t), \tag{10}$$

$$q_k = \bar{q}_k / \sum_{k=1}^{N} \bar{q}_k. \tag{11}$$

However, this introduces a sampling bias in the gradient estimates g_k from the clients which needs to be corrected before updating the local weights. Through importance sampling and bias correction [9], we obtain

$$\hat{\mathbf{g}}_{k,t+1} = \mathbf{g}_{k,t+1} * p_k / q_k \tag{12}$$

where $p_k = \frac{n_k}{\sum_{k=1}^{N} n_k}$ is the original probability of sampling client k and q_k is the importance sampling probability. Although, this approach adjusts bias for the weight after one

round, when the number of local rounds $\tau > 1$, the gradient estimate at each client after each local update is biased. Therefore, we propose to apply bias correction at the client after each local update. In other words, the local weight update for the general case of τ local updates is given by

$$\mathbf{w}_{k,t+\tau} = \mathbf{w}_{k,t+\tau-1} - \eta * \hat{\mathbf{g}}_{k,t+\tau}. \tag{13}$$

Upon receiving K updates, the weights are combined as

$$\mathbf{w}_{t+\tau} = \sum_{k=1}^{K} \mathbf{w}_{k,t+\tau}/K. \tag{14}$$

We term this approach **FedIS I** method. However, this approach alone cannot result in accelerated training as K clients sampled using q_k described above may have widely differing upload time T_k^{up} which is given as

$$T_k^{up} = T_k^{comp} + T_k^{comm}. \tag{15}$$

We characterize the compute and communication models of the clients as follows:

• ***Wireless Communication Model:*** The communication time T_k^{comm} is the time taken for client k to upload model weights to server and is given by

$$T_k^{comm} = D_{size}/r_k \tag{16}$$

where the payload D_{size} is the sum of the sizes of the ML model parameters and r_k is the communication rate of client k. We assume clients are connected through 5G wireless links to their base stations (BSs) and then to the MEC server (and assuming no additional latency from BSs to the server). r_k depends on location of the clients, the radio capability, number of antennas, etc. The details of the wireless communication model and client heterogeneties are provided in Section IV.

• ***Compute Model:*** Clients may have different processing rates, memory constraints and number of active processes. The computation time is a random variable [8]

$$T_k^{comp} = T_k^{c1} + T_k^{c2}, \tag{17}$$

with heterogeneity of compute time across clients statistically modeled using a shifted exponential distribution and given as

$$p_{T_k^{c2}(t)} = \begin{cases} \gamma_k e^{-\gamma_k * (t - T_k^{c1})}, & \text{if } t >= T_k^{c1}. \\ 0, & \text{else} \end{cases} \tag{18}$$

$T_k^{c1} = \tau * num_k^{MAC} * T_k^{MAC}$ is the deterministic component that depends on the time required per multiply-and-add (MAC) computation T_k^{MAC}, the number of MAC computations num_k^{MAC} on n_k dataset and the number of local epochs τ. The random component $T_k^{c2}(t)$ models the randomness coming from memory read/write cycles needed for computation with the rate parameter $\gamma_k = 1/(0.5 * T_k^{c1})$.

• ***Importance Sampling with Compute Communication awareness:*** The importance sampling distribution q is computed from the ratio of the normalized weighted losses to the normalized upload times given by

$$q_k = \frac{\hat{\hat{q}}_k / \hat{T}_k^{up}}{\sum_{k=1}^{N} \hat{\hat{q}}_k / \hat{T}_k^{up}}. \tag{19}$$

Algorithm 1: **FedIS** Algorithm. S: MEC server, C: Clients

Initialize N, K, τ, η, $\mathbf{w}_1$, original client sampling distribution $\mathbf{p}$
Clients report a) Communication Time T_k^{comm}, b) Compute Time T_k^{c1} to compute T_k^{up}
for all global epochs $t = 1, \tau, ..., \tau T$ **do**
    S: Broadcasts global weight $\mathbf{w}_t$
    C: Share scalar *weighted loss information* $n_k F_k(\mathbf{w}_t)$ to S (In the absence of $F_k(\mathbf{w}_t)$, S utilizes the most recently reported $F_k(\mathbf{w})$)
    S: Update T_k^{comm} as well as receive any updated T_k^{c1} from clients
    S: Compute $q_k \forall$ N clients from Eq. 11 (FedIS I) or Eq. 19 (FedIS II)
    S: Draw K clients using q_k for weight update
    for each client k = 1, 2,..., K **do**
        for each local epochs e = 1, 2,..., τ **do**
            Compute $\mathbf{w}_{k,t+e} = \mathbf{w}_{k,t+e-1} - \eta * \mathbf{g}_{k,t+e} * p_k/q_k$
        end for
        Report $\mathbf{w}_{k,t+\tau}$ to server and scalar $n_k F_k(\mathbf{w}_{t+\tau})$
    end for
    S: Compute global weight at server as $\mathbf{w}_{t+\tau} = \sum_{k=1}^{K} \mathbf{w}_{k,t+\tau}/K$
end for

Algorithm 2: **FedRO** based Fair Resource Management. S: MEC server, C: Clients

Initialize N, K, τ, η, $\mathbf{w}_1$
Clients report a) Communication Time T_k^{comm}, b) Compute Time T_k^{c1} to compute T_k^{up}
for all global epochs $t = 1, \tau, ..., \tau T$ **do**
    S: Broadcasts global weight $\mathbf{w}_t$
    C: Share scalar *weighted loss information* $n_k F_k(\mathbf{w}_t)$ to S (In the absence of $F_k(\mathbf{w}_t)$, S utilizes the most recently reported $F_k(\mathbf{w})$)
    S: Update T_k^{comm} as well as receive any updated T_k^{c1} from clients
    S: Rank order clients based on largest $n_k F_k(\mathbf{w}_t)$
    S: Select top K_1 clients greedily from the rank ordered list
    S: Sub-select K clients with shortest upload time T_k^{up}
    for each client k = 1, 2,..., K **do**
        Report $\mathbf{w}_{k,t+\tau}$ to server and scalar $n_k F_k(\mathbf{w}_{t+\tau})$
    end for
    S: Compute global weight at server as $\mathbf{w}_{t+\tau} = \sum_{k=1}^{K} \mathbf{w}_{k,t+\tau}/K$
end for

This is due to the fact that the MEC server is also interested in selecting clients with small upload times. The importance sampling bias correction is then applied as in Eq. 13 and this method is termed as **FedIS II**. The proposed importance sampling based FL approach is described in Algorithm 1.

B. Approach 2: Rank Order based Fair Resource Management

In this case, the goal is to minimize the maximum weighted loss of clients $\max n_k F_k(\mathbf{w}_{t+\tau})$ subject to compute and communication time constraints. We propose a heuristic where we select $K_1 >= K$ clients with the largest loss in each round for contention. From the selected group, K clients with the smallest T_k^{up} are selected for the global epoch. Intuitively, this approach balances the losses across clients thereby minimizing the loss variance while also accounting for the compute and communication time. The parameter K_1 determines the tradeoff between fairness and compute-communication delay of the FL. When $K_1 = K$, the algorithm ignores the compute-communication delays during client selection. On the other hand, when $K_1 = N$, the algorithm aims to reduce the time per global epoch while ignoring model fairness. We term this approach **FedRO** and summarize in Algorithm 2.

IV. Experimental Evaluation

A. Experimental Setup

We consider the image classification task and compare the performance to FedAvg and other baselines on MNIST [12], Fashion MNIST [13] and LEAF datasets [13]. The client compute, communication and data models are given below:

a) Wireless Communication Model: We consider that the clients are wirelessly connected to their base stations (BSs) (and assuming no additional latency from BSs to the MEC server). The uplink data rates of clients are obtained based on 5G deployment and channel model as described in the ITU-R M.2412 report [11]. We consider two possible deployment scenarios including a) *Dense Urban-eMBB*, and b) *Indoor Hotspot-eMBB* where the *Indoor Hotspot-eMBB* scenario is further classified into 4GHz and 30GHz operating frequencies. For the 30GHz indoor hotspot case, we also allow the clients to have either 12 or 30 transmission reception points (TRxPs). The TRxPs determine the number of vertical, horizontal antenna elements within a panel and per polarization and the number of polarizations and panels. We allow single-user MIMO and multi-user MIMO operation in all these scenarios.

The uplink communication rates of clients are determined by their deployment scenario (Indoor Hotspot or Urban), operating frequency and their antenna configuration (number of antenna elements and MIMO modes). Each user is uniform randomly assigned one of the above configurations and the location of the users are randomly sampled from the possible range which is based on the specific deployment scenario. For example, in the case of indoor hotspot, client's location is selected uniform randomly in a $120m \times 50m$ rectangular grid containing 12 BSs. Based on this, the average data rate of each client is obtained as r_k. A 32-bit representation is utilized for the payload D_{size} containing the weight matrices.

b) Compute Model: The mean MAC rate μ_0 of the best case client is at 40% of the peak MAC rate of 384 TMACs. The average MAC rates for other clients are computed as $\mu_k = 0.9^k * \mu_0$. Using this, the fixed part of compute time is given as $T_k^{c1} = num_k^{MAC}/\mu_k$ which is shared by clients to the MEC server. The random component T_k^{c2} is not available to the MEC server for resource allocation decisions and is only utilized in the performance evaluation.

c) Statistical Model: For MNIST and Fashion MNIST, we simulate two cases where the data distribution between clients are not I.I.D. This includes (i) **Case 1:** Each client has only 1 class of image leading to highly-skewed data distribution, (ii) **Case 2:** 20% of clients have I.I.D samples of the overall distribution, while the remaining 80% clients have 2 to 6 classes with their mean centered at 4 classes. In both cases, all the clients have equal number of training data. For LEAF dataset specifically curated for federated learning use cases, we utilize FEMNIST dataset built by partitioning data in Extended MNIST based on the writer of the digit/character. Each client contains different number of training data.

d) Implementation: We utilize Tensorflow to implement the baselines FedAvg, FedProx [5] as well as our proposed resource management algorithms for training a deep neural network (DNN) with 2 hidden layers each consisting of 200 neurons. The hidden layers utilize ReLu activations and the output layer returns the softmax probabilities over the image classes. For FedProx, we utilize a regularizer term $\frac{\mu}{2}||\mathbf{w}-\mathbf{w}_t||^2$ to the loss function with $\mu = 2$. For each global epoch, we run τ local epochs and compare the test performances under different setting. The test data follows the same distribution as the overall training data. We evaluate the test accuracy after 2000 epochs and obtain the model with best performance in the last 10 epochs for each of the algorithms.

For each global round, the proposed algorithms require the computation of $F_k(\mathbf{w}_t)$ which results in additional compute and communication overheads. Hence, we only utilize the $F_k(\mathbf{w}_{t-\tau})$ that clients compute from previous global epoch. In fact, we also compare the performance of the proposed methods when utilizing $F_k(\mathbf{w}_{t-2\tau})$ in some cases. Hence, the additional overhead of the proposed algorithms originates from the computation and upload of $F_k(\mathbf{w}_{t-\tau})$ as well as the periodic update of T_k^{comm} and T_k^{c1}. In order to capture different client compute and communication rates, we train different DNNs with 3 different random seeds and compute the average performance for each of the approaches. It must be noted that we clip the multiplier p_k/q_k to [0.25, 2] to ensure the gradients do not become too large or too small in the Eq. 12 for our importance sampling method.

TABLE I: Evaluation Parameters

Parameter	Value	Parameter	Value
Total clients N	100, 105	Clients/round K	1, 10
Global epochs	2000	Local epochs τ	1,5
Training algo	SGD	Learning rate η	0.01
Batch size B	50	Validation split	0.2
Channel model	5G eMBB	Carrier Freq.	4, 30GHz
MIMO modes	SU, MU MIMO	TRxP	12 or 30
Max. MAC rate	384 TMACs	Avg. Utilization	40%

B. Performance Results

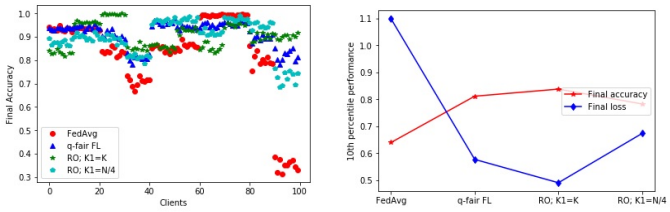
Table II shows the test performance. For MNIST and Fashion MNIST, Case 1 points to a highly skewed data distribution across clients leading to lower overall training accuracy compared to Case 2. FedIS I approach has better accuracy and test loss over FedAvg and FedProx in both cases except for $\tau = 1$ in Case 2. FedIS II approach can beat FedAvg and FedProx when $\tau = 5$ while achieving $4.5x$ reduction in the convergence time, but drops marginally (to within 0.75%) when $\tau = 1$ but reducing the convergence time by $10x$. This is a significant speedup for FL where devices inherently differ in compute and communication rates. It is also important to note that we only utilize partial compute time of clients, since T_k^{c2} is a random component and not available at MEC server when determining the sampling distribution $\mathbf{q}$. For LEAF dataset, FedIS I's accuracy is either as good or better than FedAvg and FedProx, while FedIS II achieves over $4x$ reduction in convergence time for 1% loss in accuracy. We therefore note the tradeoff between the training time and loss/accuracy performance.

TABLE II: Approach 1: MNIST, Fashion MNIST, LEAF

Metric	**FedAvg**	**FedProx**	**FedIS I**	**FedIS II**
MNIST, Case 1, N=100, K=10, τ=5				
Accuracy	90.56%	90.68%	**96.72%**	95.92%
Loss	0.30	0.31	**0.11**	0.13
Tot. Time (hours)	12.75	12.75	12.4	**2.84**
MNIST, Case 1, N=100, K=10, τ=1				
Accuracy	91.68%	91.69%	**91.76%**	90.93%
Loss	0.28	0.28	**0.28**	0.31
Tot. Time (hours)	2.43	2.43	2.6	**0.25**
MNIST, Case 2, N=100, K=10, τ=5				
Accuracy	97.21%	97.22%	**97.94%**	97.80%
Loss	0.09	0.09	**0.07**	0.08
Tot. Time (hours)	12.75	12.75	11.51	**2.81**
MNIST, Case 2, N=100, K=10, τ=1				
Accuracy	**95.97%**	95.97%	95.87%	95.13%
Loss	**0.13**	0.13	**0.13**	0.16
Tot. Time (hours)	2.43	2.43	2.37	**0.25**
Fashion MNIST, Case 2, N=100, K=10, τ=1				
Accuracy	85.89%	**85.92%**	85.58%	85.04%
Loss	**0.40**	0.40	**0.40**	0.42
Tot. Time (hours)	2.43	2.43	2.35	**0.23**
LEAF, N=105, K=10, τ=5				
Accuracy	76.89%	**77.45%**	77.37%	75.98%
Tot. Time (hours)	8.19	8.19	7.84	**2.09**

TABLE III: Approach 2: Model Fairness Performance

MNIST, Case 1, N=100, K=10, τ=5				
Metric	**FedAvg**	**q=2 fair**	**FedRO** K_1=K	**FedRO** K_1=N/4
Mean Acc. (local)	82.69%	**91.24%**	90.39%	89.79%
Acc. Variance	0.03	0.002	**0.002**	0.005
10^{th}%ile Acc.	64%	81.16%	**83.79%**	78.28%
Mean Loss (local)	0.52	**0.29**	0.30	0.32
90^{th}%ile loss	1.09	0.57	**0.49**	0.67
Tot. Time (hours)	12.75	13.12	14.09	**2.68**
LEAF, N=105, K=10, τ=5				
		q=5 fair		
Accuracy	**78.33%**	76.82%	77.05%	75.67%
Acc. Variance	0.034	0.027	**0.024**	0.027
10^{th}%ile Acc.	51.81%	58.33%	**58.33%**	56.06%
Tot. Time (hours)	8.19	8.4	6.54	**1.91**



(a) Accuracy on Local Test Data (b) 10^{th} %ile performance

Fig. 2: Approach 2: Model Fairness Performance

For FedRO based client selection, we implemented Algorithm 2 under the same setting as above on MNIST with Case 1 non-IID dataset. Each client has 20% of data withheld as test dataset and the accuracy and loss are computed at the end of training on this dataset. We compare the variance and $10^{th}\%ile$ performance of accuracy and loss for all the clients to measure fairness. We have also benchmarked the proposed solutions with a q-fair federated learning approach [7]. To mitigate the overhead of the proposed fair resource management approach, we utilize the loss information $F_k(t-\tau)$ from previous epoch. The q-fair FL approach requires additional computation for Hessian approximation.

Table III shows the overall performance for MNIST and LEAF datasets. Both $q-fair$ and FedRO approach improve fairness by reducing variance of accuracy and loss and the FedRO approach in particular has the best performance for the worst case 10% of clients. We also note that $K_1 = K$ focuses strictly on reducing the variance of the losses and therefore achieving higher fairness performance. As K_1 is relaxed to $N/4$, the 10^{th} percentile performance is worse than both q-fair and $K_1 = K$ case but outperforming FedAvg, but more importantly reducing convergence time by $4.5x$. This highlights the tradeoff achievable between convergence time and model fairness. Figure 2 shows a more detailed view of the accuracies and losses for MNIST case.

V. Conclusion

The problem of resource management in federated learning over wireless edge networks was formalized in this paper. To address this, importance sampling and rank ordering based algorithms were developed that allow prioritization of clients based on their resources in the presence of non-I.I.D data across clients. Performance evaluation on a supervised image classification task on benchmark datasets showed significant reduction in the overall training time without loss of performance in the model training compared to state of the art approaches. Fair resource allocation approach was also developed that not only improves model fairness but also to tradeoff fairness with total training time.

References

[1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas. "Communication-efficient learning of deep networks from decentralized data", In Conference on Artificial Intelligence and Statistics, 2017.
[2] T. Nishio, R. Yonetani, "Client Selection for Federated Learning with Heterogeneous Resources in Mobile Edge", IEEE ICC, May 2019.
[3] H. H. Yang, Z. Liu et al, "Scheduling Policies for Federated Learning in Wireless Networks", arXiv:1908.06287, Oct. 2019.
[4] M. Chen, Z. Yang et al, "A Joint Learning and Communications Framework for Federated Learning over Wireless Networks", arXiv:1909.07972, Sep. 2019.
[5] T. Li et al, "Federated Optimization for Heterogeneous Networks", Conference on Machine Learning and Systems (MLSys), 2020.
[6] C. Dinh et al, "Federated Learning over Wireless Networks: Convergence Analysis and Resource Allocation", arXiv:1910.13067, Nov. 2019.
[7] T. Li, M. Sanjabi, V. Smith, "Fair Resource Allocation in Federated Learning", ICLR, 2020.
[8] S. Dhakal, S. Prakash et al, "Coded Computing for Distributed Machine Learning in Wireless Edge Network," IEEE Vehicular Technology Conference, USA, 2019, pp. 1-6.
[9] I. Goodfellow, Y. Bengio, A. Courville, "Deep Learning", The MIT Press, ISBN:978-0-262-03561-3, November 2016
[10] T. Schaul, J. Quan, I. Antonoglou, D. Silver, "Prioritized Experience Replay", ICLR 2016.
[11] ITU-R, Report ITU-R M.2412-0. Guidelines for evaluation of radio interface technologies for IMT-2020, October 2017.
[12] Y. LeCun, L. Bottou, Y. Bengio, P. Haffner. "Gradient-based learning applied to document recognition", Proceedings of the IEEE, 86(11):2278–2324, 1998.
[13] H. Xiao et al, "Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms", arXiv:1708.07747, Sep. 2017.
[14] S. Caldas, S. M. K. Duddu et al, "LEAF: A Benchmark for Federated Settings", arXiv:1812.01097, Dec. 2019.